3. Data Collections – Tuples, Dictionaries, Lists, Strings

Total: **43 questions**
Need: **24 questions**

Question 2:

What will the output be, if we run the following code?

```
1 | dict1 = {1:"One", 2:"Two"}
2 | dict1[2] = "One"
3 | print(dict1)
```

- ○ `{1: One, 2: Two}`
- ○ `No Output`
- ○ `{1: "One",1 : "One"}`
- ○ `{1:"One", 2:"One"}`

D

Question 8:

What will the output be, if we run the following code?

```
1 weekdays = ("Monday","Tuesday","Wednesday","Thursday")
2 weekdays.append("Friday")
3 print (weekdays)
```

- ○ `Friday`
- ○ `None`
- ○ `Monday, Tuesday, Wednesday, Thursday`
- ○ `AttributeError: 'tuple' object has no attribute 'append'`

D – if I switch the parenthesis () for the array creation with Square bracket [], I then will be able to append (add) value to it.
list literals – arrays that can hold different data types as elements

Question 10:

Will the following code run without errors?

```
1 tup1 = (1,3,5)
2 tup2 = (2,4)
3
4 tup1 = tup1+tup2
5 print(tup1)
```

- ○ **This code will not run.**
- ○ **This code will run without errors.**

B – it prints 1, 3, 5, 2, 4

Question 20:

What will the output be after executing the following code?

```
Tupl = ['Python', 'Tuple']
print(tuple(Tupl))
```

- ○ `(Python, Tuple)`
- ○ `['Python', 'Tuple']`
- ○ `('Python', 'Tuple')`
- ○ `[Python, Tuple]`

C –
tuple – is one of the four built-in data types in python. it is used to store multiple items in a single variable.
if it was just printing arrays *print(Tupl)* – it prints **[2, 3]**
if it tries to print a **tuple** function and pass array as parameter, -- it prints **('Python', Tuple')**

Question 24:

What will the output be after running the following code?

```
def oddoreven(num):
    if (num % 2 == 0):
        print('even')
    else:
        print("odd")

oddoreven(13)
```

- ○ `odd`
- ○ `even`

A -

Start

Question 1:

What will the output be after running the following code snippet?

```
1 lst = ["apples","bananas", ""]
2 lst.remove("apples")
3 print(lst)
```

- ○ `['bananas']`
- ○ `['apples']`
- ○ `['apples', 'bananas', '']`
- ○ `['bananas', '']`

D – it removes the element and the related index

Question 2:

What do you expect the following code snippet to printout:

```
1 tupl = tuple('Python World!')
2 print(tupl[:-7])
```

- ○ `('n', 'o', 'h', 't', 'y', 'P')`
- ○ `[P, y, t, h, o, n]`
- ○ `('W', 'o', 'r', 'l', 'd', '!')`
- ○ `('P', 'y', 't', 'h', 'o', 'n')`

D – starts from first char and ends in position -7 (or 6)
(start_position:end_position)
since it is a tuple, each character becomes an element with its own quotation. strings need quotes, numbers don't!

★ Question 4:

What will the output be after running the following code snippet?

```
1 | nums = [1, 2, 3, 4, 5, 6, 7]
2 | print(nums[::-1])
```

- ○ `[2, 3, 4, 5, 6, 7]`
- ○ `[1, 2, 3, 4, 5, 6]`
- ○ `[7, 6, 5, 4, 3, 2, 1]`
- ○ `[1, 2, 3, 4, 5, 6, 7]`

C – start from last position and go backwards
nums[:-N] – means, start from first index and end on to the indicated position
numbs[::-N] – means, start from the indicated positon and go backwards

★ Question 7:

What will the output be after running the following code snippet?

```
1 | t1 = (1, 2, 3)
2 | t2 = ('apples', 'banana', 'pears')
3 | print(t1 + t2)
```

- ○ `('apple, 'banana', 'pears', 1, 2, 3)`
- ○ `(1, 2, 3, 'apples', 'banana', 'pears')`
- ○ `(1, 2, 3), ('apples', 'banana', pears')`
- ○ `(1, 2, 3) + ('apple', 'banana'. 'pears')`

B

★ Question 14:

What will the output be after running the following code snippet?

```
def myfun(num):
    if num >= 4:
        return num
    else:
        return myfun(1) * myfun(2)

print(myfun(4))
```

- ○ `1`
- ○ `4`
- ○ `0`
- ○ `2`

B

★ Question 15:

What do you expect the following code to print:

```
nums = [1, 2, 3, 4]
nums.append(5)
print(nums)
```

- ○ `[1, 2, 3, 4, 5]`
- ○ `[1, 2, 3, 4]`
- ○ `[5, 4, 3, 2, 1]`
- ○ `[5, 1, 2, 3, 4]`

A

Question 21:

What will the output be after running the following code snippet?

```
d = {'one':2,'two':2}
d['one'] = 1
print(d)
```

- ○ `{'one': 0, 'two': 1}`
- ○ `{'one': 0, 'two': 2}`
- ○ `{'one': 1, 'two': 0}`
- ○ `{'one': 1, 'two': 2}`

D

Question 2:

What will be the output after running the following code?

```
numbers = dict([('first', 3),('second', 1),('third', 2)])
print(numbers.pop('second'))
```

- ○ `[('first', 3),(1),('third', 2)]`
- ○ `[('first', 3),('third', 2)]`
- ○ `1`
- ○ `second`

C

numbers is a dict (object). it holds one element that is a list (editable array).

Explanation
The `pop()` method was passed 'second' as an argument. Hence, it will fetch the value associated with the key `second`, which is `1`.

Python list pop() is **an inbuilt function in Python that removes and returns the last value from the List or the given index value**. Syntax: list_name.pop(index) Parameter: index (optional) – The value at index is popped out and removed. https://www.geeksforgeeks.org/python-list-pop/#:~:text=Python%20list%20pop()%20is,is%20popped%20out%20and%20removed.

Question 12:

What will the output be after executing this code?

```
x = []
y = ""
z = -1

print(bool(x),bool(y),bool(z))
```

- False True False
- True True False
- False False False
- False False True

D

Explanation
When `bool()` is provided an empty list, an empty string or the value 0 as an argument, it returns False. The expression will evaluate to `True` when `bool ()` is given any other value, including negative numbers.

bool(empty string or value) = False
bool (any number including negative) = True

Question 13:

What will be the output after running the following code?

```
languages = {'lang1': {1: 'Python'},
             'lang2': {2: 'Java'}}
print (languages['lang1'][1])
```

- ○ `Java`
- ○ `1`
- ○ `error`
- ○ `Python`

D

Explanation

`languages` represents a dictionary of a dictionaries. The `dict['lang1']` returns the value of the key *lang1*, which is a dictionary, and `dict['lang1'][1]` returns the first **value** in the retrieved sub-dictionary `{1: 'Python'}`, which is `Python`.

Question 14:

What is the output of the following code:

```
lst = [1, 2] * 5
print(len(lst))
```

- ○ `error`
- ○ `10`
- ○ `9`
- ○ `5`

B

just like when multiplying a string by number repits that word the number of times, multiplying array by number repeats its elements that much

[1,2]*5 = [1,2,1,2,1,2,1,2,1,2], len = 10

Explanation
The list contains two elements, and when multiplied by 5, we are repeating the same elements 5 times. Therefore the length of the new list is `10`.

★ Question 18:

What will be the output when the following program is run?

```
1 | tupl = 5,4,"Earth"
2 | print(list(tupl))
```

- ○ `[5, 4, 'Earth']`
- ○ `5,4,'Earth'`
- ○ `[5,4]`
- ○ `{5,4,'Earth'}`

A

Explanation
The `list()` function will convert the tuple `5,4,"Earth"` into a list object.

★ Question 19:

What will be the output after running the following code?

```
1  tuple_one = (1, 2, 3)
2  tuple_two = ("Apples", "Bananas")
3  tuple_three = (tuple_one + tuple_two)
4  print(tuple_three)
```

- ○ `SyntaxError`
- ○ `(1, 2, 3, 'Apples', 'Bananas')`
- ○ `(1, 2, 3)('Apples', 'Bananas')`
- ○ `('Apples', 'Bananas', 1, 2, 3)`

B

Explanation

Adding two tuples together will produce a new tuple, which is a combination of the original tuples.

★ Question 20:

What will be the output after running the following code?

```
1  val = ['Python', 'Tuple']
2  val_t = tuple(val)
3  val_t.pop()
4  print(val_t)
```

- ○ `['Tuple']`
- ○ `[]`
- ○ `['Python']`
- ○ `AttributeError`

D
what is tuple.**pop**
.pop is a method that can takeout and get the last element of the array, or that can get an element of an object if provided the index of that element.
However, pop() method can't work on tuple because tuple is uneditable array!

Explanation
This code will produce an error, since tuples are immutable and cannot be changed once created. The `pop()` method cannot be used on tuples.

Question 23:

What is the output when the following code is executed:

```
1 vowels = ["a", "e", "i", "o", "u"]
2 all =  list(range(-2)) + vowels
3 print(all)
```

- ○ `['o', 'u']`
- ○ `['a', 'e', 'i', 'o', 'u']`
- ○ `['a', 'e', 'i']`
- ○ `['a', 'e', 'i', 'o', 'u', 'a', 'e', 'i', 'o', 'u']`

B

Explanation
The `range(-2)` does not generate any numbers. Hence, `all` is assigned the list of vowels, and printed.

Question 26:

What will be the output after running the following code?

```
1 nums = [1, 2, 3, 4, 5, 6, 7]
2 print(nums[::-5])
```

- ○ `[7, 3]`
- ○ `[7, 2]`
- ○ `SyntaxError`
- ○ `[7, 6, 5, 4, 3, 2]`

B

Explanation

The data from the list is accessed in reverse and includes every 5th element starting from the back of the list.

Other examples:

`print(nums[::-2])` will print `[7, 5, 3, 1]`

`print(nums[::-3])` will print `[7, 4, 1]`

★ Question 6:

What will be the output after running the following code?

```
1 tupl1 = (-1, 0, 1)
2 tupl2 = ('bananas')
3 tupl3 = (tupl1, tupl2)
4 print(tupl3)
```

- ○ `(-1, 0, 1, 'bananas')`
- ○ `((-1, 0, 1), 'bananas')`
- ○ `(-1, 0, 1)('bananas')`
- ○ `('bananas', (-1, 0, 1))`

B – it used a comma (not addition +) so it wont join them. It creates nested tuple

Explanation

The `tupl3` was created using two tuples, however we did not use the concatenation operator. Instead, tupl1 and tupl2 were enclosed in a tuple `(tupl1, tupl2)`. This creates a nested tuple .

★ Question 10:

What will be the output after running the following code?

```
1 d = {}
2 d[0] = 'Python'
3 d['weekends'] = ["Saturday","Sunday"]
4 print(d)
```

- ○ `{'Python', ["Saturday", "Sunday"]}`
- ○ `['Python', 'weekends', ['Saturday', Sunday']]`
- ○ `{0: 'Python', 'weekends': ['Saturday', 'Sunday']}`
- ○ `IndexError`

C

Explanation
We are inserting data into our dictionaries by assigning values to the keys.

Question 14:

What will be the output after running the following code?

```
a = [1, 2, 3]
a.append(2)
a.append(1)
print(a)
```

- ○ `[1, 1, 2, 2, 3]`
- ○ `error`
- ○ `[1, 2, 3, 2, 1]`
- ○ `[1, 2, 3, 1, 2]`

C

Explanation
The `append()` method adds the data at the end of the list, so 2 and 1 are added in succession.

Question 17:

What will be the output after running the following code?

```
dict1 = {"John":1234, "Fruit":"Apples"}
dict2 = {"Fruit":"Apples", "John":1234}
print(dict1 == dict2)
```

- `not equal`
- `False`
- `equal`
- `True`

D -- they are equal (not equivalent). same number of key/val combinition

Explanation

The dictionaries store data in key-value format, hence the comparison between two dictionaries is on the basis of key value pairs having the same positions, and not solely on the values.

Question 23:

What is the output when the following program is executed?

```
data = [1, 2, "apples", 3.14, True]
del data[:2]
print(data)
```

- `[3.14, True]`
- `[1, "apples", 3.14, True]`
- `[1, 2, "apples"]`
- `['apples', 3.14, True]`

Explanation

The `del` operator deletes the elements, in this case, from the beginning of the list until the element at position 1 (excluding 2).

Question 28:

What will be the output after running the following code?

```
a = ["Monday", "Wednesday", "Thursday"]
a.insert(1,"Tuesday")
a.append("Friday")
print(a)
```

- ○ `[ "Monday", "Tuesday, "Wednesday", "Thursday", "Friday"]`
- ○ `["Tuesday","Friday", "Monday", "Wednesday", "Thursday"]`
- ○ `["Tuesday", "Monday", "Wednesday", "Thursday", "Friday"]`
- ○ `["Monday", "Wednesday", "Thursday"]`

Explanation

The `insert()` method takes two values: an index and a value. It then inserts the value in the list at the specified position.

The `append()` method will add the element at the end of the list.

Question 29:

What will be the output after running the following code?

```
1 tupl = ('Python','World') * 2
2 print(tupl)
```

- ○ `error`
- ○ `('Python', 'World', 'Python', 'World')`
- ○ `('Python', 'World')('Python', 'World',)`
- ○ `('Python', 'World', 2)`

Explanation

The tuple when multiplied by an integer produces a new tuple with the same elements, multiplied X number of times (in this example, twice).

Question 9: **Skipped**

What will be the output of the following print statements?

```
1 lst1 = [2,4,6]
2 lst2 = [2,4,6]
3
4 print(lst1 is lst2)
5 print(lst1 == lst2)
```

○

```
False
True
```

(Correct)

○

```
True
True
```

○

```
False
False
```

○

```
True
False
```

Explanation

The `is` operator is an identity operator, it checks whether both the operands refer to the same object or not. lst1 and lst2 are two different objects (despite their content being the same).

The `==` operator checks if the values stored in the variables are equal to each other, which is `True` in this case.

Question 12: **Skipped**

What is the output of the following code snippet:

```
1 fruits = ("Apples", "Oranges", "Bananas")
2 a, b, c = fruits
3 print(b)
```

- ○ `Oranges` **(Correct)**
- ○ `'Apple', 'Oranges', 'Bananas'`
- ○ `Bananas`
- ○ `error`

Explanation

The elements in a tuple can be unpacked and assigned to variables as shown here. In this example, we are declaring 3 variables, to which we are assigning the tuple. The first element will be assigned to a, the 2nd to b, and the 3rd to c. Hence the print statement prints out `Oranges` .

Note that the number of variables must be the same the number of elements in the tuple, otherwise the Python interpreter will throw a `ValueError: not enough values to unpack`

Question 13: **Skipped**

What will be the output of the following print statement?

```
1 tupl1 = (1., 2., 3.)
2 tupl2 = ("Earth", "Mars", "Jupiter")
3 x = (tupl1 + tupl2)[-3]
4 print(x)
```

- ○ `3`
- ○ `error`
- ○ `1.,2.,3.`
- ○ `'Earth'` **(Correct)**

Explanation

Adding two tuples will result in a new tuple with the values from both tuples combined.

We then extract the element value at position -3, the third position starting from the back of the list. This element is 'Earth'.

Question 14: Skipped

Which of the following statements is correct :

- [ ] **A. A tuple cannot be modified once created, as they are immutable by design**
- [ ] **B. Adding tuples will produce a new tuple**
- [ ] **C. The elements in a tuple can be accessed by using their index position**
- [ ] **D. All of the above** (Correct)

Explanation

Tuples are immutable by design and hence the data cannot be changed once created. Adding multiple tuples together will produce a new tuple with all the elements combined. The elements can be accessed by its index.

Question 15: **Skipped**

What is the result of the following print statement?

```
1 groceries_list1 = ["Milk", "Cheese"]
2 groceries_list2 = ["Bread", "Butter"]
3 groceries_list1.extend(groceries_list2)
4 print(groceries_list1)
```

- ○ `['Milk', 'Cheese']`
- ○ `['Bread', 'Butter']`
- ○ `['Milk', 'Cheese', 'Bread', 'Butter']` **(Correct)**
- ○ `SyntaxError`

Explanation

The `extend()` method used on a list adds the second list to the main list, producing a super set of elements.

Question 16: **Skipped**

What is the result of the following print statement?

```
1  capitals1 = ["London","New York","Rome"]
2  capitals2 = capitals1
3  capitals2.remove("New York")
4  print(capitals1)
```

- ○ `"New York"`
- ○ `['London', 'Rome']` **(Correct)**
- ○ `["London","New York","Rome"]`
- ○ `SyntaxError`

Explanation

Both variables point to the same list values. Changing one variable changes the underlying list.

Question 17: **Skipped**

What would be the output of the following code when executed?

```
dict = {1:"iOS"}
dict[2] = 'Android'
print(dict)
```

- ◯ `{1: 'iOS', 2: 'Android'}` **(Correct)**
- ◯ `{1: 'iOS', 'Android'}`
- ◯ `{'iOS', 'Android'}`
- ◯ `{1: 'iOS'}`

Explanation

The dictionary was created with a key-value pair of `1: 'iOS'`. We can add a new key-value pair in the following manner: `dict['key'] = value`

Question 18: **Skipped**

What will be the output of the following code when executed?

```
cities = {"UK":"London","France":"Paris","Germany":"Berlin"}

for city in cities.items():
    print(city)
```

○

```
London
Paris
Berlin
```

○

```
UK
France
Germany
```

○

```
('London', 'UK')
('Paris', 'France')
('Berlin', 'Germany')
```

○

```
('UK', 'London')
('France', 'Paris')
('Germany', 'Berlin')
```

(Correct)

Explanation

The `items()` method used on a dictionary fetches the set of key-value pairs. In this example, we will get all the sets as country-capital tuples printed out.

Question 6: **Skipped**

What is the output of the following code snippet?

```
address_book = {'name': "John", 'age': 23, 'city': "London"}

while address_book:
    address_book.popitem()
print(address_book)
```

- ◯ `{}` **(Correct)**
- ◯ `SyntaxError`
- ◯ `{'name': "John", 'age': 23}`
- ◯ `{''age': 23, 'city': "London"}`

Explanation

The `popitem()` method deletes the last element in the series. Hence all the items are deleted as the loop iterates through the elements in `adress_book`, resulting in an empty dictionary.

Question 7: **Skipped**

What will be the output when this code is executed?

```
1 | ui_elements = dict([('radio_button', 2),('text_box', 3),('standard_button',
  | 5)])
2 | popped_element = ui_elements.popitem()
3 | print(list(popped_element))
```

- ◯ `['standard_button', 5]` **(Correct)**
- ◯ `[('radio_button', 2),('text_box', 3)]`
- ◯ `['radio_button', 2]`
- ◯ `error`

Explanation

The dictionary is constructed by using tuples enclosed in a list. Popping an item from the dictionary will remove last element. Here we are capturing the last element in a variable by using `popitem()`, converting the element to a list and printing it out to the console.

Question 8: **Skipped**

What is the output of the following code when executed?

```
1  me = "apples",
2  print(type(me))
```

- ○ **Invalid syntax since a comma shouldn't be there**
- ○ `<class 'str'>`
- ○ `error`
- ○ `<class 'tuple'>` **(Correct)**

Explanation

In the first line, a single element with a comma will be created as a tuple. Hence, the data type of `me` is a tuple.

Question 11: **Skipped**

What will be the result of the following print statement?

```
1 high_fives = [10, 0, 5, 15]
2 high_fives.sort(reverse=True)
3 print(high_fives)
```

- ○ `[15, 5, 0, 10]`
- ○ `SyntaxError`
- ○ `[15, 10, 5, 0]` **(Correct)**
- ○ `[0, 5, 10, 15]`

Explanation

The `sort()` method on a list will sort the elements in ascending order.

However, should you pass `reverse=True`, the list will then be sorted in descending order. Hence, the resulting list goes from larger to smaller values: `[15, 10, 5, 0]`

Question 15: **Skipped**

What will be the output of the following print statements?

```
1 ones = [1]
2 ones_again = ones.extend([11,111])
3 print(ones)
4 print(ones_again)
```

- ○ `error`
- ○ `[1, 11, 111]`
- ○ `[11, 111, 1]`
- ○

```
1 [1, 11, 111]
2 None
```

(Correct)

Explanation

The `extend()` method used on a list adds the elements of the specified list provided as an argument to the end of the current list. Hence ones stores `[1, 11, 111]` and is printed out by the first print statement.

However, the `extend()` function does not return anything, it operates on the list on which it was used (`ones`). Hence `ones_again` stores the value of `None`.

Question 19: **Skipped**

What will be the output when this code is executed?

```
1 lst = [x for x in range(3)]
2 print(lst)
```

- ○ `[3]`
- ○ `[0, 1, 2, 3]`
- ○ `[0, 1, 2]` **(Correct)**
- ○ `[1, 2, 3]`

Explanation

This is a list comprehension, a way to define and create lists based on existing lists. We are creating a list by selecting elements from the range 0 up to 3, using a `for` loop. Each of the elements in this range is collated to make up the new list.

Question 21: **Skipped**

What will be the output of the following print statement?

```
1 tupl = ("bananas", "apples", "cherries")
2 print(sorted(tupl))
```

- ○ `['apples', 'bananas', 'cherries']` **(Correct)**
- ○ `['bananas', 'apples', 'cherries']`
- ○ `['cherries', 'bananas', 'apples']`
- ○ `SyntaxError`

Explanation

The `sorted()` function sorts the given data structure alphabetically.

The result is the original list, sorted in alphabetical order.

Question 27: **Skipped**

What will be the output when the following code is executed?

```
1 fruits = ("Apples", "Oranges", "Bananas")
2 fruit1, fruit2, fruit3 = fruits
3 print(fruit3)
```

- ○ `Oranges`
- ○ `('Apples', 'Oranges')`
- ○ `error`
- ○ `Bananas` **(Correct)**

Explanation

The tuples can be unpacked into equivalent number of variables as shown in the example above. Each element in the tuple will be stored in one of the variables, in that order. Hence `fruit1` stores `Apples`, `fruit2` stores `Oranges`, and `fruit3` stores `Bananas`.

Question 30: **Skipped**

What is the output of the following code?

```
1  fruits = ["apples","cherries"]
2
3  for fruits[-1] in fruits:
4      print(fruits[-1], end ="|")
```

- ◯ `apples|apples|` **(Correct)**
- ◯ `apples|cherries|`
- ◯ `IndexError`
- ◯ `apples|cherries`

Explanation

In the for loop, the first iteration fetches "apples", right? Usually we set this element on to a local variable - like for f in fruits where the f is the local variable. But in this case what we are doing is a bit weird - we are setting it to a variable called fruits[-1]. Now what is fruits[-1]. Isn't this accessing the element from the fruits list at -1th position? Exactly! That is, we are actually setting the first element fetched during the first iteration ("apples") to fruits[-1] which is nothing but the last element - this means we are replacing the "cherries" to "apples"!! We are actually MODIFYING the main list by running this for loop and assigning it to that local variable!

Section 3: Data Collections – Tuples, Dictionaries, Lists, Strings

3.1 – Collect and process data using lists
- constructing vectors
- indexing and slicing
- the len() function
- list methods: append(), insert(), index(), etc.
- functions: len(), sorted()
- the del instruction
- iterating through lists with the for loop
- initializing loops
- the in and not in operators
- list comprehensions
- copying and cloning
- lists in lists: matrices and cubes

3.2 – Collect and process data using tuples
- tuples: indexing, slicing, building, immutability
- tuples vs. lists: similarities and differences
- lists inside tuples and tuples inside lists

3.3 Collect and process data using dictionaries
- dictionaries: building, indexing, adding and removing keys
- iterating through dictionaries and their keys and values
- checking the existence of keys
- methods: keys(), items(), and values()